Running SpringBoot Application in Docker

1. pre-requisite – Docker Desktop

- Before running SpringBoot application:
  - Make sure **Docker Desktop** is installed.
  - Make sure Docker Desktop is **running** in the **background**
- **Docker commands** require **Docker Desktop/Docker Engine** to be running
- Do **not shutdown** docker desktop while working with docker.

2. Check available Docker Images

- Displays the list of **Docker Images available locally.**
- Example
  - Spring Boot application's image
  - Paketo Buildpacks – related images
  - Other images previously downloaded or built

Command

‹/› Bash

```
docker images
```

Remember

> `docker images` → Shows **images** on your local machine.

3. Check Running Containers

- **Ps -** Displays the list of **currently running containers**
- Typically shows:
  - Container ID
  - Image
  - Command
  - Created Time
  - Status
  - Ports
  - Container Name

Command

Bash

```
docker ps
```

Remember

`docker ps` → Shows **running containers.**

4. Run the Spring Boot Image as a Container

Command

```bash
docker run -p 8080:8080 <image-name>
```

Example:

```bash
docker run -p 8080:8080 username/ecom-application
```

What happens?

- `docker run`:
  - Creates a **new container** from the specified image.
  - Starts the container.
- `<image-name>`:
  - Specifies which Docker image should be used.

5. Understanding –p 8080

Bash

```
-p 8080:8080
```

means:

```
LOCAL MACHINE          DOCKER CONTAINER
     8080       →           8080
```

- First `8080` → port on your **local/host machine**
- Second `8080` → port inside the **Docker container**

So when you access:

```
http://localhost:8080
```

the request is forwarded to:

```
Container → port 8080
```

Easy way to remember

Bash

```
-p HOST_PORT:CONTAINER_PORT
```

Example:

Bash

```
-p 9090:8080
```

means:

```
localhost:9090 → container:8080
```

6. Port already in use error

You may get an error such as:

```
port is not available
```

This usually means the host port you're trying to use is already occupied.

For example:

Bash

```
-p 8080:8080
```

but another application is already using **port 8080**.

Solution

- Stop the application currently running on port `8080`.
- Then run the Docker container again.

Or choose another host port:

Bash

```
docker run -p 9090:8080 <image-name>
```

Now:

```
localhost:9090 → container:8080
```

7. Running container in Normal/Foreground mode

If you run:

```bash
docker run -p 8080:8080 <image-name>
```

- The container runs in the **foreground**.
- You can see the application's output/logs directly in the terminal.
- That terminal is occupied by the running container.
- Pressing:

```
Ctrl + C
```

stops the foreground container.

8. Run container in Detached Mode

To avoid blocking the terminal, use:

```bash
docker run -d -p 8080:8080 <image-name>
```

`-d` **means:**

> **Detached mode**

- Container runs in the **background**.
- Your terminal remains available.
- Docker returns a **container ID**.

Example:

```bash
docker run -d -p 8080:8080 username/ecom-application
```

Output:

```
a7f3c9......
```

That long value is the **container ID**.

9. Verify Container is Running

After starting the container:

Bash

```
docker ps
```

You should see your container.

Example:

```
CONTAINER ID   IMAGE                STATUS
a7f3c9...      ecom-application     Up ...
```

Remember

```
docker images → Images
docker ps     → Running containers
```

10. View container logs

- When running in detached mode, we don’t see the logs directly in our terminal.
- **Logs** can show:
  - Spring Boot startup messages
  - Application status
  - SQL Queries
  - Errors
  - Exceptions
  - Other application output
- This is extremely useful for **troubleshooting**

Bash

```
docker logs <container-id>
```

Example:

Bash

```
docker logs a7f3c9
```

You can also use the container name if you gave it one:

Bash

```
docker logs my-app
```

11. Test the Spring Boot APIs

- Once the container is running:
  - Open **Postman**
  - Call your Spring Boot APIs using the mapped host port
  - Example : localhost:8080
- Create a user
- Create a product
- Get products
- Check the response

If SQL logging is enabled in the application, you can see the SQL queries in:

```bash
docker logs <container-id>
```

Example:

```
insert into products ...
```

This confirms that the Spring Boot application is running successfully inside the container.

12. Stop the container

- First find the container using **docker ps**
- Then stop the container using **docker stop** command
- The container stops
- The container itself **still exists**
- We can **start** it again **later**

Bash

```
docker stop <container-id>
```

Example:

Bash

```
docker stop a7f3c9
```

13. Start a stopped container

- If we previously stopped a container, we don’t need to create a new one

Use:

Bash

```
docker start <container-id>
```

or:

Bash

```
docker start <container-name>
```

Important difference

```
docker run   → Creates + starts a NEW container
docker start → Starts an EXISTING stopped container
```

14. Check after stopping

Run:

```bash
docker ps
```

- The stopped container will **not** appear because `docker ps` only shows running containers.

To see it:

```bash
docker ps -a
```

This shows:

- Running containers
- Stopped containers

15. Docker Image vs Container

This is very important for interviews:

```
Docker Image
     ↓
 docker run
     ↓
Docker Container
```

Image

- Blueprint/template.
- Contains the application and everything required to run it.

Container

- Running instance created from an image.

One image can be used to create multiple containers.

16. If Image is not available locally

If you don't have the image locally, first download it from a registry.

Command

Bash

```
docker pull <image-name>
```

Example:

Bash

```
docker pull username/ecom-application
```

Then verify:

Bash

```
docker images
```

You should see the image locally.

Then run:

Bash

```
docker run -p 8080:8080 username/ecom-application
```

Complete flow

```
Docker Registry
      ↓
 docker pull
      ↓
Local Docker Image
      ↓
 docker run
      ↓
Docker Container
      ↓
Spring Boot Application
```

17. Docker Desktop

- Docker Desktop provides a graphical interface where you can see
- **Images –** Locally available Docker Images.
- **Conatiners**
  - Running containers
  - Stopped containers
  - Container logs
  - Container statistics and the other information
- You can also stop containers from Docker Desktop instead of using

```bash
docker stop <container-id>
```

18. cleaning-up Docker Resource

- Docker Images can consume significant disk space
- We can remove a container after stopping it

‹/› Bash

```
docker rm <container-id>
```

And remove an unused image:

‹/› Bash

```
docker rmi <image-name>
```

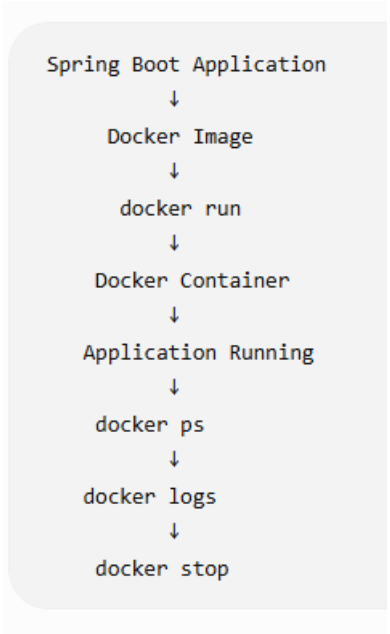
Important

```
docker rm   → removes a CONTAINER
docker rmi  → removes an IMAGE
```

Most Important commands from this lecture

Command	Purpose
`docker images`	List local images
`docker ps`	List running containers
`docker ps -a`	List all containers
`docker pull <image>`	Download image
`docker run <image>`	Create & run container
`docker run -d <image>`	Run container in background
`docker run -p 8080:8080 <image>`	Map host port to container port
`docker logs <container>`	View container logs
`docker stop <container>`	Stop container
`docker start <container>`	Start stopped container
`docker rm <container>`	Remove container
`docker rmi <image>`	Remove image

Interview Memory Flow



One-line interview explanation

- I containerize my Spring Boot application into a Docker Image
- Run that image as a container using **docker run**
- Map the containers application port to a host port using –p
- Use commands like below to manage the container
  - Docker ps
  - Docker logs
  - Docker stop
  - Docker start